Grounded Local AI for University Accreditation

Alfa Yohannis
Faculty of Science and Technology
Pradita University, Tangerang, Indonesia
alfa.ryano@pradita.ac.id

***Abstract*—Preparing a university accreditation submission means answering hundreds of detailed questions, and the evidence for each is buried somewhere in a shared folder of thousands of files. This paper describes a working system that lets staff ask a question in everyday Indonesian and receive an answer in seconds, in which every statement carries a numbered citation pointing to the exact file and page it came from. It is built on retrieval-augmented generation: the institution's own documents are searched first, and the language model may write only from what was found. We explain the architecture in plain terms and the design choices that make the answers checkable—source files are never modified, and the whole system runs on one office workstation, so confidential material need not leave the building.**

***Index Terms*—retrieval-augmented generation, document question answering, accreditation, hybrid retrieval, large language models, higher education**

I. The Problem

An accreditation submission is an exercise in evidence. For every claim a study programme makes—about its curriculum, its lecturers, its graduates, its research—an assessor may ask where the proof is. The proof exists: in a decree issued three years ago, in row 47 of a spreadsheet, on page 12 of a curriculum document. It is simply spread across thousands of files accumulated over years in a shared drive that several colleagues edit at once.

The consequence is familiar: a single question can cost half an hour of opening files one by one, and under deadline pressure the tempting shortcut is to answer from memory—which is how an unsupported claim reaches an assessor. A general-purpose chatbot does not help, having never seen these documents yet still producing a fluent answer that cites nothing. What is needed is a system that answers *only* from the institution's own files, and shows its working.

II. What the User Sees

The system is a web page that opens in any browser on the office network, including a phone, offering a single box for a question in Indonesian. Seconds later an answer appears in ordinary prose, a small numbered marker beside each statement, and beside it an evidence panel: one card per marker giving the document's name, the page or sheet the passage came from, and the passage itself. Clicking a card opens the original file. The user is therefore never asked to trust the system—every sentence can be checked in one click.

Asked whether the programme teaches machine learning, for instance, the system replies that the curriculum lists a *Machine Learning and Deep Learning* course, and attaches the curriculum spreadsheet and the sheet its course table sits on, so the reader can open it and see the row.

III. Architecture

Fig. 1 shows the system twice over: panel (a) follows a question from the shared folder to a cited answer, panel (b) shows the same system as components. The backend is Python; the browser interface is plain JavaScript, with no framework and no build step.

A. Reading the documents

The system walks the shared drive and opens every file it understands—PDF, Word, Excel, PowerPoint, web pages, plain text, and scanned images through optical character recognition—and pulls the words out. Crucially, it remembers *where* each piece of text was: page 12, sheet "Dosen", slide 4. That bookkeeping is what makes a citation possible later.

Evidence on public websites—the ministry's database, research profiles, staff pages—is fetched once, saved as a dated copy, and thereafter indexed like any other document. Long documents are split into passages of a few hundred words that overlap slightly at the edges, so a fact straddling a boundary is never cut in two. Re-running the process re-reads only files that actually changed, detected from each file's size and modification time, so a daily update takes seconds rather than the tens of minutes a full pass costs.

B. Two kinds of search

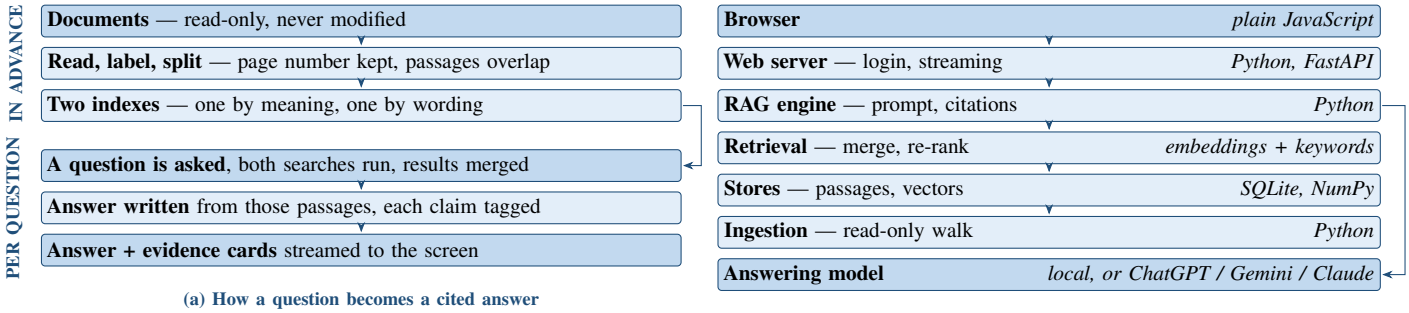
Every passage is filed in two separate indexes, because the two ways of searching fail in different places.

Search by meaning converts each passage into a list of numbers—an embedding [3]—capturing what the text is about. The question is converted the same way, and the closest passages are returned. This understands that "lecturer teaching load" and the abbreviation "EWMP" mean the same thing though they share no words, but it is unreliable with exact identifiers.

Search by exact wording is ordinary keyword search [4]. It finds a decree number such as `SK-045/XYZ/VI/2025` precisely, and is blind to paraphrase.

Accreditation documents are full of both: narrative argument on one page, decree and registration numbers on the next. Dropping either method would quietly make a whole class of questions unanswerable.

The two result lists cannot simply be added, because their scores are on different scales. The system merges them by



(a) How a question becomes a cited answer

(b) Components, and what each is built with

Fig. 1. Architecture of the system. In panel (a) the upper half runs whenever documents change and the lower half runs in seconds each time somebody asks. Arrows point in the direction work flows; note in panel (b) that the answering model is reached only through the RAG engine, and never touches the document store itself.

position instead: a passage ranked highly by both methods is promoted above one only a single method favours. This is reciprocal rank fusion [2], and it needs no retuning when the collection changes. The two dozen survivors are then read once more, in full, against the question by a second, slower model that scores question–passage pairs directly, and only the best handful is carried forward.

C. Writing the answer

Only those final passages are handed to a language model, with an instruction to answer from them alone and to tag each statement with the number of the passage supporting it. This arrangement—search first, then write only from what was found—is retrieval-augmented generation [1]. The model cannot browse the web, open files or run commands, and is instructed to say so when the passages do not contain the answer. The evidence cards reach the browser *before* the first word of the answer, so the reader can start checking sources while it is still being written.

D. Choosing the model

The answering model is interchangeable, and the choice is a privacy decision more than a technical one. By default an 8-billion-parameter model [5] runs *locally*, so no document text leaves the building; for harder questions the operator may select ChatGPT, Gemini or Claude, which synthesise better but send the retrieved passages outside. The interface states the trade-off on the selector itself, so the choice is made knowingly.

IV. Design Decisions

Source documents are never modified. The shared drive is the real evidence archive, edited daily, so the rule is enforced in software: every file operation passes through a guard that refuses to open a source file for writing, and the server refuses to start if its own working folder sits inside the archive. A test verifies that a full indexing pass leaves every file byte-for-byte identical.

External evidence is frozen, not fetched. Public pages change, so the system cites its dated copy rather than the live page—while still linking to it—so that an assessor sees the page that was actually quoted.

Answers survive lost connections. Answers are produced on the server independently of the requesting browser, with every event numbered, so a user whose connection drops resumes exactly where the text left off.

V. Deployment and Scale

The system is in use for a Master's programme accreditation submission. The index covers **1,497 documents**—about 188,000 sentences and 3.5 million words. It runs on one desktop—a 20-core Core i7-14700F, 32 GB of memory, an RTX 4060 with 8 GB of video memory—and 7.4 GB of disk, mostly the downloaded models. Answers take seconds to under a minute. That single machine serves the whole team, with a full-privilege account for the administrator and a read-only account for everyone else.

VI. Conclusion

The contribution is not a new algorithm but a demonstration: an accreditation team can have a question-answering assistant whose answers are *checkable by construction*—built from its own documents, citing them by page, leaving them untouched, and running on hardware the department already owns. The approach transfers to any organisation that must justify claims from a large, messy, authoritative archive.

References

[1] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020, pp. 9459–9474. [Online]. Available: https://arxiv.org/abs/2005.11401

[2] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd ACM SIGIR Conf.*, 2009, pp. 758–759. [Online]. Available: https://doi.org/10.1145/1571941.1572114

[3] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "M3-Embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," arXiv:2402.03216, 2024. [Online]. Available: https://arxiv.org/abs/2402.03216

[4] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009. [Online]. Available: https://doi.org/10.1561/1500000019

[5] A. Yang *et al.* (Qwen Team), "Qwen3 technical report," arXiv:2505.09388, 2025. [Online]. Available: https://arxiv.org/abs/2505.09388